

De um número de 16 bits até a Web inteira.

Toda a P1 cabe em uma frase: **a camada de transporte inventa a porta, e a camada de aplicação gasta esse endereço construindo serviços**. Primeiro, como duas aplicações se acham e conversam com (TCP) ou sem (UDP) garantia; depois, os serviços clássicos que rodam em cima disso: terminal remoto, arquivos, nomes, correio e Web. Esta é a versão condensada do guia completo: definições, tabelas, fórmulas e as respostas-chave dos 30 exercícios do professor.

10	30	2	35%
módulos	exercícios do professor cobertos	fórmulas (RTT e RTO)	peso da P1 na média

Se o tempo for curto: módulos 1 a 6 valem mais, porque as questões de aplicação (DNS, SMTP, HTTP) sempre voltam a falar de porta, conexão TCP e papel cliente-servidor. Os exercícios mais prováveis estão nos blocos vermelhos de cada módulo.

Sumário

TRANSPORTE

1 · Transporte: papel e primitivas

2

2 · Portas e multiplexação

2

3 · UDP

3

4 · TCP: conexão, header e flags

4

5 · Confiabilidade e controle de fluxo

5

6 · Congestionamento e RED

6

APLICAÇÃO

7 · Telnet, SSH e FTP

7

8 · DNS

8

9 · Correio: SMTP, MIME, POP3, IMAP

9

10 · Web: HTTP, URL e Proxy

10

FERRAMENTAS

Laboratório: comandos essenciais

11

Números, portas e RFCs

12

O que NÃO cai

13

Provável na prova

Comando / código

☐ Exemplo resolvido

1 · Camada de transporte: papel e primitivas

Visão **fim-a-fim**: as camadas de baixo levam bits de máquina a máquina; a de transporte leva dados de **aplicação a aplicação**. É a **4ª camada do OSI** e a **3ª do TCP/IP** (o TCP/IP funde física + enlace embaixo e sessão + apresentação + aplicação em cima). Interface de acesso: os **sockets** (SAP = Service Access Point).

- **Dois papéis na Internet**: (1) identificação precisa e não-ambígua de cada aplicação — a **porta**; (2) entrega de mensagens entre cada par de aplicações, segundo parâmetros de QoS.
- **Encapsulamento**: TPDU (segmento) → **pacote** na camada de rede → **quadro** na camada de enlace. Cada camada tem sua PDU; trocar os nomes é erro clássico.

Os dois eixos de um serviço de transporte

Eixo	Significado	Internet
Confiabilidade	Confirmação, retransmissão e não duplicação	TCP sim; UDP não
Conexão	Circuito virtual fim-a-fim; dados ordenados, fluxo contínuo (<i>stream</i>)	TCP sim; UDP <i>connectionless</i>

Só duas das quatro combinações existem: TCP (conexão + confiável) e UDP (sem conexão + não confiável).

As cinco primitivas

Primitiva	Quem	O que faz
LISTEN	servidor	Fica em espera por conexões
CONNECT	cliente	Requisita conexão a quem está em LISTENING
SEND	ambos	Envia dados
RECEIVE	ambos	Espera bloqueada até chegar dado
DISCONNECT	cliente (o correto)	Encerra a conexão

Paradigma cliente-servidor

	Servidor	Cliente
Papel	Passivo ("escravo"): aguarda requisição → executa o serviço → envia resposta	Ativo ("mestre"): envia requisição → aguarda resposta

Quatro possibilidades de interação (caem em cenário): um cliente acessa vários serviços (browser usa HTTP e FTP); um cliente acessa vários servidores do mesmo serviço (página com imagens em outro servidor); **servidores viram clientes de outros servidores** (MTA e DNS); um host roda várias aplicações clientes e servidoras ao mesmo tempo.

Provável na prova

- **L1 q.1 — vantagem do TCP por ser orientado à conexão**: circuito virtual fim-a-fim preestabelecido → transmissão ordenada e confiável (confirmação, retransmissão, não duplicação); a aplicação vê um *stream*. O UDP não reordena nem garante chegada.
- **L1 q.4 — navegador busca imagens em outro servidor**: possibilidade 2; o cliente abre uma conexão por recurso, cada relação identificada pelo próprio par (IP, porta).
- Enumerar as 5 primitivas com quem chama cada uma; desenhar TPDU → pacote → quadro.

2 · Portas, multiplexação e identificação de aplicações

- **Multiplexação**: várias aplicações compartilham a mesma entidade de transporte. Na entrada, a porta de destino demultiplexa para a aplicação certa.
- **Porta**: identificador único **local**, inteiro de **16 bits** (65.536 valores, 0-65535, separados por protocolo: 53/TCP ≠ 53/UDP). O servidor a usa para se registrar; o cliente a informa como destino. A porta do cliente é escolhida pelo SO na faixa alta (efêmera).

• **Identificação global de uma conexão = quatro números:**

(IP, porta) do servidor + (IP, porta) do cliente — o *socket pair*.

Faixa (IANA, RFC 3232)	Nome	Uso	Exemplo
< 1024	Well Known Ports	Reservadas ao sistema; exigem root (proteção contra falsificação de serviço)	80 → HTTP
> 1023	Registered Port Numbers	Registradas, mas utilizáveis por aplicativos de usuário	3306 → MySQL

Duas abas no mesmo servidor

Cliente 192.168.0.10 → servidor 200.1.1.5:80. Aba 1 = (192.168.0.10, 49152)↔(200.1.1.5, 80); aba 2 = (192.168.0.10, 49153)↔(200.1.1.5, 80). Três números iguais; a **porta de origem** distingue as conexões — por isso "a porta 80" sozinha nunca identifica uma comunicação.

Provável na prova

- **L1 q.2:** identificação pela porta associada ao IP; campo de 16 bits no TCP e no UDP (origem e destino).
- **L1 q.5/q.6:** navegador + e-mail + vídeo no mesmo host, ou HTTP + MySQL na mesma máquina → uma entidade de transporte, portas distintas, demultiplexação pela porta de destino (80 → HTTP, 3306 → MySQL).
- **L1 q.7 (a mais provável):** só a porta não basta porque ela é única apenas **dentro do host**; a conexão exige IP e porta de origem + IP e porta de destino.
- **L1 q.8:** HTTP na 8080 → clientes precisam informar a porta na URL (`http://host:8080/`); em compensação, o servidor sobe sem root.

3 · UDP (User Datagram Protocol) — RFC 768

Serviço de **datagrama**, semelhante ao IP: **não orientado à conexão e não confiável**; não reordena e não garante chegada. Vantagem: simplicidade e **pouco overhead**. É o **único que permite broadcast local**; use-o quando é melhor "perder" do que parar a aplicação — o TCP "serve para todo o resto".

Campo (16 bits cada)	Conteúdo
Source port	Porta de origem
Destination port	Porta de destino
UDP length	Total em bytes: header + dados
UDP checksum	Opcional; se não calculado, zeros

4 × 2 bytes = **header de 8 bytes** (contra 20 do TCP). No Linux: struct udphdr, campos __be16 em *big endian* (ordem de bytes da rede).

Campo len

Consulta DNS com 29 bytes de dados →

UDP length = 8 + 29 = 37 . Pegadinha: responder "29".

	RPC (RFC 1057)	RTP (RFC 1889)
Para quê	Chamada de procedimento remoto que parece local	Fluxos multimídia (<i>streams</i>)
Mecanismo	Client stub / server stub (implementações vazias); o RPC intercepta e manda pela rede; acha o servidor pelo portmapper	Acrescenta ao UDP identificação de fluxo, timestamp e sincronização; <i>best-effort</i>
Por que UDP	Chamada e resposta curtas, sem conexão; o próprio RPC reenvia se estourar timeout	"Vale mais perder alguns pixels do quadro do que atrasar sua exibição"

Provável na prova

- **L1 q.9 — por que o UDP continua amplamente usado:** baixo overhead (8 bytes), sem custo de conexão, menor latência (não espera ACK), aplicações em que perder é preferível a atrasar (streaming, voz) ou que tratam a perda sozinhas (DNS, RPC), e broadcast local.
- Listar os 4 campos e tamanhos; explicar RPC (stubs, portmapper) ou RTP; calcular len.

4 · TCP: conexão, header e flags — RFC 793

Principal protocolo de transporte da Internet. Serviço de **stream: orientado à conexão e confiável**, transmissão ordenada por uma conexão preestabelecida, mantida durante o serviço e encerrada ao final (o correto é o cliente pedir). Limites do PDU: payload máximo do IP = **65.515 bytes** (65.535 – 20 de header) e, na prática, o **MTU Ethernet de 1.500 bytes** → 1500 – 20 (IP) – 20 (TCP) = **1460 bytes de dados** (MSS típico).

Header TCP (20 bytes mínimo, até 60)

Campo	Bits	Função
Source / Destination port	16 + 16	Portas
Sequence number	32	Numera os bytes enviados ($2^{32} \approx 4,29$ bi)
Acknowledgement number	32	Próximo byte esperado (confirma até ali)
Data Offset	4	Tamanho do header em palavras de 32 bits
Reserved	4	—
Flags	8 × 1	CWR ECE URG ACK PSH RST SYN FIN
Window size	16	Janela do controle de fluxo
Checksum	16	Integridade
Urgent pointer	16	Offset dos dados urgentes
Options + Padding	var.	Opções; padding completa 32 bits

As oito flags

Flag	Significado
CWR	Congestion Window Reduced: emissor reduziu a janela de congestionamento
ECE	ECN-Echo: houve congestionamento no caminho
URG	Urgent pointer em uso; dados urgentes (interrupção, ex.: Ctrl+C)
ACK	Campo Acknowledgement válido (em quase todo segmento)
PSH	Entregar cada mensagem que chega, sem esperar acumular (interativas)
RST	Reset: conexão com problema / porta fechada → recusa abrupta
SYN	Solicitar/aceitar estabelecimento (só nos 2 primeiros segmentos)
FIN	Solicitar/aceitar encerramento negociado (um de cada lado)

RST × FIN: FIN = "terminei, confirme"; RST = "essa conexão não existe, descarte". Servidor sem aplicação na porta responde RST.

Tamanho do header

bytes = Data Offset × 4 — doff 5 → 20

(mínimo); doff 8 → 32 (12 de options); doff 15 → 60 (máximo, 4 bits). Dados = total length IP – header IP – doff×4.

Three-way handshake

Servidor em LISTENING com ACCEPT bloqueante. (1) Cliente CONNECT → **SYN=1, ACK=0**. (2) Se a porta não aceita → **RST=1**; se aceita → **SYN=1, ACK=1**. (3) Cliente confirma → **ACK=1**. Conexão aberta. Encerramento é semelhante, com FIN no lugar de SYN. São três vias porque a conexão é **full-duplex**: cada

sentido anuncia e confirma seu número de sequência inicial (que não é zero, para não confundir segmentos atrasados de conexões anteriores).

#	Sentido	Flags	Campos (x = 1000, y = 5000)
1	cliente → servidor	SYN=1, ACK=0	SEQ = 1000
2	servidor → cliente	SYN=1, ACK=1	SEQ = 5000, ACK = 1001
3	cliente → servidor	ACK=1	SEQ = 1001, ACK = 5001

Regra do +1: o ACK aponta para o próximo byte esperado; o SYN consome um número de sequência mesmo sem dados.

UDP x TCP: a tabela que resolve toda questão de escolha

Critério	UDP (RFC 768)	TCP (RFC 793)
Conexão / confiabilidade	Sem conexão; sem confirmação nem retransmissão; não reordena	Three-way handshake; confirmação positiva com retransmissão; entrega ordenada (stream)
Header / overhead	8 bytes, 4 campos; baixíssimo	20-60 bytes; alto (handshake, ACKs, timers, buffers)
Controle de fluxo / congestionamento	Nenhum	Janela deslizante / janela de congestionamento
Broadcast local	Só ele	Impossível
Usos em aula	RPC, RTP, DNS	Telnet, SSH, FTP, SMTP, POP3, IMAP, HTTP

Provável na prova

- **L2 q.1:** explicar o estabelecimento da conexão — os passos acima com as flags de cada segmento (vale desenhar).
- **L1 q.10 / L2 q.3:** finalidade de SYN, ACK, FIN e RST; cenário "servidor sem aplicação na porta": SYN=1/ACK=0 → recebe RST=1.
- **L2 q.5:** SEQ numera os bytes enviados; ACK confirma até onde chegou e pede o próximo; Window Size diz quanto ainda cabe no buffer do receptor.
- **L1 q.3 / q.11 (escolha):** banco e controle industrial → TCP (integridade e ordem); vídeo ao vivo e câmeras → UDP (atraso é pior que perda). Justificar por confiabilidade, atraso e overhead.

5 · Confiabilidade, RTT/RTO e controle de fluxo

Confirmação positiva com retransmissão: o destinatário envia ACK; o originador inicia um relógio e, se expira antes do ACK, reinicia e retransmite. Só se confirma o que chegou — a ausência de ACK (timer) sinaliza a perda. Se o **ACK se perde**, o emissor retransmite, o receptor reconhece pelo **número de sequência** que aqueles bytes já foram entregues, descarta a cópia e reenvia o ACK: nada se perde nem se duplica.

Timeout adaptativo (Algoritmo de Retransmissão Adaptativo)

Timeout fixo não serve: alto demais reduz eficiência; deve ser por conexão, dinâmico, menor na LAN que em 20 saltos. Baseia-se no **RTT** (Round Trip Time) e define o **RTO** (Retransmission Timeout):

$$RTT_{\text{new}} = (\alpha \times RTT_{\text{old}}) + ((1 - \alpha) \times RTT_{\text{sample}})$$

$$RTO = \beta \times RTT_{\text{new}}$$

Média ponderada: alpha alto → estável, reage devagar; alpha baixo → reage rápido, oscila. $\beta > 1$ dá folga para não retransmitir cedo demais.

Exemplo

$\alpha = 0,875 \cdot RTT_{old} = 100 \text{ ms} \cdot RTT_{sample} = 140 \text{ ms} \cdot \beta = 2 \rightarrow RTT_{new} = 0,875 \times 100 + 0,125 \times 140 = 87,5 + 17,5 = \mathbf{105 \text{ ms}}$; $RTO = 2 \times 105 = \mathbf{210 \text{ ms}}$. A amostra subiu 40 ms, a estimativa só 5 ms: a média amortece.

Controle de fluxo: janela deslizante

O **receptor** especifica o tamanho da janela (**window advertisement**), transportado no ACK: confirma o próximo byte esperado e informa quantos bytes ainda aceita. Exemplo do slide (buffer de 4 KB): emissor manda 2K (SEQ 0) → ACK 2048, WIN 2048 → manda mais 2K → **ACK 4096, WIN 0** (buffer cheio: emissor bloqueado, não é erro) → aplicação lê 2K → ACK 4096, WIN 2048 → emissor pode mandar até 2K. É sempre o receptor quem reabre a janela.

Síndrome da janela boba (Silly Window Syndrome)

Se a aplicação destino consome poucos bytes, o receptor anuncia janelas minúsculas e a origem as preenche a cada vez → muitos segmentos pequenos, banda desperdiçada (1 byte útil por 40 de headers ≈ 2,4% de eficiência). **Soluções:** o **destinatário** atrasa o anúncio de janela livre até valer a pena; a **origem** atrasa o envio até receber janela satisfatória.

Provável na prova

- **L2 q.2:** o tamanho de janela = bytes que ainda cabem no buffer de recepção; exerce o **controle de fluxo**.
- **L2 q.4:** ACK perdido — detecção pelo timer de retransmissão; número de sequência evita perda e duplicação.
- **L2 q.6:** transmissor rápido × receptor lento → janela deslizante com window advertisement, chegando a WIN = 0.
- **L2 q.7:** conexão na LAN × entre continentes → RTT e RTO muito diferentes; valor fixo seria ineficiente numa e causaria retransmissões na outra → algoritmo adaptativo.
- **L2 q.8:** síndrome da janela boba e as duas estratégias. Aplicar as fórmulas com números do enunciado.

6 · Controle de congestionamento e RED

O TCP detecta congestionamento por dois sinais: o **RTT das confirmações subindo** (filas nos roteadores) e **confirmações não recebidas** (timeout = descarte por fila cheia). A **janela de congestionamento** começa mínima (mensagens pendentes sem confirmação), cresce gradativamente enquanto o RTT médio cai, e é **reduzida drasticamente** ao detectar congestionamento por timeout — podendo voltar a crescer. Sobe devagar, cai de vez (gráfico dente-de-serra). O emissor envia o menor entre a janela do receptor e a de congestionamento.

	Controle de fluxo	Controle de congestionamento
Problema	O receptor não dá conta de processar/armazenar	A rede (roteadores) não dá conta de encaminhar
Quem sinaliza	O receptor, explicitamente	Ninguém — o emissor infere
Informação usada	Campo Window size do header	RTT médio das confirmações e timeouts
Variável	Janela anunciada pelo receptor	Janela de congestionamento, interna ao emissor — não existe campo no header (só CWR/ECE dão dicas)

RED — Random Early Detection / Discard / Drop

Explora a janela de congestionamento: o roteador, ao ver a fila crescer, **descarta pacotes de propósito, antes de encher, escolhendo fluxos aleatoriamente**; cada emissor atingido reduz sua janela, aliviando a rede antes do colapso. Sem RED, a fila transborda de uma vez, todos recuam e crescem juntos

(sincronização global). A perda é o único "canal" do roteador com o TCP. Programas "confrontam" o RED abrindo muitas conexões paralelas (perdem só uma fração) ou usando UDP (sem janela para reduzir).

Provável na prova

- **L2 q.9 (quase certa):** comparar fluxo × congestionamento — problema que cada um resolve e informações usadas (a tabela acima é o roteiro).
- **L2 q.10:** servidor com muitos clientes em horário de pico — (a) janela de congestionamento cresce/encolhe por RTT e timeout; (b) RED descarta seletiva e aleatoriamente antes da fila encher; (c) programas confrontam com conexões paralelas ou UDP.
- Significado de cada letra de RED e por que o descarte é aleatório; interpretar o gráfico dente-de-serra.

7 · Camada de aplicação: Telnet, SSH e FTP

Um protocolo de aplicação deve especificar **duas coisas**: o **formato das mensagens (PDUs)** e o **comportamento das entidades cliente e servidora** nas diversas situações (sintaxe e significado). Serviços tradicionais são especificados por **RFCs (IETF)** e têm portas registradas pela **IANA** (/etc/services). Contexto: anos 60 terminais burros; anos 70-80 transferência de arquivos, correio e terminal remoto (FTP, SMTP, Telnet); anos 90 a Web; 1994 os ISPs.

Telnet — RFC 854, porta 23/TCP

Emulação de terminal virtual via **NVT** (Network Virtual Terminal): cliente ↔ telnetd. Full-duplex de caracteres; cada caractere é enviado ao servidor e ecoado de volta. **Problema:** sessão, login e senha trafegam em **texto plano** → sniffers. Hoje serve como cliente genérico de protocolos texto (SMTP 25, POP3 110).

SSH — RFCs 4250-4254, porta 22/TCP

Emulação de terminal **segura**: autenticação segura, integridade e encriptação. Modelo híbrido: **RSA (assimétrico)** autentica e combina a chave de sessão; **3DES/AES (simétrico, chave gerada pelo cliente)** cifra o fluxo, mais barato. Também cria **túneis** seguros para outras aplicações (arquivos, HTTP, SMTP). Na 1ª conexão o cliente aceita a chave pública do servidor.

FTP — RFC 959, portas 21 e 20/TCP

Transferência de arquivos ASCII ou binário por **duas conexões TCP**: **21 = canal de controle** (comandos, respostas, autenticação; aberto a sessão toda) e **20 = canal de dados** (conteúdo; aberto e fechado a cada transferência). Separar permite mandar comandos (abortar) durante a transferência. Autenticação, acesso anônimo, put/get, mput/mget.

	Modo ATIVO	Modo PASSIVO
Início	Cliente conecta na porta 21	
Passo 2	Cliente informa a porta em que receberá dados	Cliente avisa que o modo é passivo
Canal de dados	Servidor abre a conexão em direção ao cliente	Cliente abre a conexão com a porta 20 do servidor
Firewall	Problemas: conexão de entrada não solicitada é bloqueada	Atravessa sem dificuldade (padrão moderno)

Provável na prova

- **L4 q.1 (quase certa):** FTP usa duas portas para separar controle (21) de dados (20); ativo × passivo = quem abre o canal de dados (servidor × cliente), e por isso o ativo trava em firewall.
- **L4 q.2 (quase certa):** Telnet = terminal virtual (NVT) sobre TCP/23; vulnerável por transmitir tudo em claro; SSH resolveu com RSA + integridade + encriptação com chave de sessão simétrica.
- Portas e RFCs de cada um; papel de IETF e IANA; as duas coisas que um protocolo de aplicação especifica.

8 · DNS (Domain Name System) — RFCs 1034 e 1035, porta 53

IPs de 32 bits são eficientes para o TCP/IP mas ruins para memorizar e não têm referência geográfica. O DNS traduz nomes ↔ IPs. **FQDN** (Fully-Qualified Domain Name): alfanuméricos separados por pontos; **esquerda = host, direita = domínio e hierarquia** (ex.: idg.receita.fazenda.gov.br). Implementação difundida: **BIND** (ISC).

- **TLDs** definidos por autoridade global (InterNIC): por atividade (.com, .edu, .gov) e geográficos (.br, .fr, .ca). Uma organização registra seu domínio sob um TLD e cria seus subdomínios.
- **Zona de autoridade:** região da hierarquia sob a autoridade de um servidor (um domínio, subdomínio ou grupo). **Domínio ≠ zona:** se fazenda.gov.br delega receita a outro servidor, um domínio vira duas zonas. Domínios são lógicos, não físicos.
- **Registro:** nome, tipo (RR) e valor, ex.: servidor IN A 192.168.0.1. Classe **IN** = Internet. **TTL** = tempo de vida em segundos, inteiro de **32 bits**.

RR	Nome	Faz
A	Address	Nome → IP
CNAME	Canonical Name	Alias de um nome
MX	Mail eXchanger	Servidores de e-mail do domínio
NS	Name Server	DNS autoritativo do domínio
PTR	Pointer	Resolução reversa IP → nome (zona <code>in-addr.arpa</code> , octetos invertidos)
SOA	Start of Authority	Início da zona (serial = versão do arquivo)
HINFO	Host Info	CPU e SO do host

Cliente-servidor

DNS request (o nome) é enviada por um **resolver** (cliente; pode ser uma API como `gethostbyname`); **DNS reply** (o IP) vem do servidor **autoritativo**. Regra: se o nome é da sua zona, responde direto; senão, **busca iterativa:** raiz → quem responde pelo TLD → próximo nível → ... → autoridade. Um DNS pode atuar como resolver em nome dos clientes e manter **cache** pelo TTL — ao fazê-lo, **torna-se cliente** dos servidores da hierarquia (possibilidade 3 do módulo 1). Iterativa = "não sei, pergunte àquele"; recursiva = devolve só a resposta final; o DNS local faz as duas.

Exercício do professor: resolver `vacinapira.piracicaba.sp.gov.br` em servidor recém-iniciado

(cache vazio, sem autoridade). (1) Resolver do cliente → DNS local. (2) DNS local: não é sua zona, não está no cache → vira cliente. (3) Pergunta à **Raiz** → NS de .br. (4) .br → NS de gov.br. (5) gov.br → NS de sp.gov.br. (6) sp.gov.br → NS de piracicaba.sp.gov.br. (7) Servidor autoritativo → registro **A** com o IP. (8) DNS local guarda em cache (TTL) e envia a reply. Cinco consultas iterativas = cinco níveis abaixo da raiz; "recém-iniciado" proíbe a resposta "busca no cache".

Arquivo de zona do BIND (prática da aula 5) — a teoria em texto

```
zone "exemplo.com.br" { type master; file "/etc/bind/db.exemplo.com.br"; }; # named.conf.local
$TTL 604800
@      IN SOA  servidor.exemplo.com.br. root.servidor.exemplo.com.br. ( 202609081 604800 86400 2419200 604800 )
      IN NS   servidor                      ; autoritativo
      IN MX   00 servidor                    ; e-mail
servidor IN A   192.168.N.1
www      IN CNAME servidor                  ; apelido
# zona reversa db.N.168.192: 1 PTR servidor.exemplo.com.br.
# cliente /etc/resolv.conf: nameserver 192.168.N.1 # testes: host www.exemplo.com.br · host 192.168.N.1
```

Provável na prova

- **L4 q.3 (praticamente certa):** os oito passos acima, explicitando cache vazio e o DNS local virando cliente da hierarquia.
- Tabela de RRs (associe colunas); zona de autoridade e domínio ≠ zona; FQDN, resolver, TTL (32 bits, segundos), classe IN; ler um trecho de arquivo de zona.
- Se perguntarem transporte: porta 53; consultas comuns por UDP (leveza, resposta cabe em um datagrama).

9 · Correio eletrônico: SMTP, MIME, POP3 e IMAP

Serviço de transferência de mensagens de **texto (ASCII)** em formato de carta, definido pela **RFC 2821 (SMTP)**. Mensagens vão para caixas postais (*mail boxes*) identificadas por `login_name@host_name`. Formato da mensagem: **RFC 822** (campos From, To, Cc, Bcc, Date, Sender, Subject, Reply-To, X-Mailer...).

Sigla	Papel
UA User Agent	O cliente: acessa caixas locais e encaminha pelo servidor (pine, mutt, webmail; remotos via POP3/IMAP: Thunderbird, Outlook)
MTA Message Transfer Agent	O servidor: encaminha e armazena nas caixas postais (sendmail, exim, postfix); porta 25

Encaminhamento por um MTA (3 passos)

1. O MTA de origem extrai o host de destino e busca no **DNS os registros MX**.
2. Encaminha aos MX por **ordem de preferência, do menor valor para o maior** (gmail: 5 → 10 → 20 → 30 → 40; menor número = mais preferido).
3. O MTA de destino extrai o usuário e armazena na caixa postal.

O MTA que encaminha **torna-se cliente** do mail server de destino, guarda cópia até a recepção completa (*store-and-forward*) e envia uma só cópia por host de destino, mesmo com vários destinatários.

MIME — RFCs 2045 a 2049

Extensão ao SMTP que permite conteúdos não-ASCII (binários, acentos): o UA anexa partes, informa o **tipo MIME**, o conteúdo é **codificado em ASCII (base64, ~33% maior)** e inserido no corpo, **delimitado e identificado** por metadados. O SMTP continua transportando só texto. Cabeçalhos: MIME-Version, Content-Type (ex.: multipart/mixed; boundary=...), Content-Transfer-Encoding, Content-Description, Content-Id. Por cuidar da **representação** dos dados, associa-se à **camada de apresentação** do OSI; virou base do HTTP.

SMTP — porta 25: os cinco comandos

HELO	Apresentação do cliente ao servidor
MAIL FROM:	Caixa postal de origem
RCPT TO:	Caixa postal de destino
DATA	Início da mensagem; termina com um "." sozinho na linha
QUIT	Encerra a conexão

Respostas numéricas: 2xx ok, 3xx continue (354 = digite a mensagem), 4xx/5xx erro.

POP3 (RFC 1939, porta 110) × IMAP v4 (RFC 3501, porta 143)

A caixa postal pode estar em outra máquina → segundo protocolo, só para **acessar** a caixa (o SMTP só envia).

	POP3	IMAP
Faz	Lista, baixa (<i>download</i>) e remove mensagens; nasceu para o dial-up: baixar e desconectar	Uso contínuo e multiplataforma; modos conectado e desconectado
Clientes	Um por vez	Múltiplos clientes simultâneos na mesma caixa
MIME	Baixa a mensagem inteira	Acessa partes MIME distintas (só o anexo)
Estado / pastas / busca	No cliente; só INBOX; busca após baixar tudo	No servidor : lida/respondida, múltiplas pastas, busca no servidor
Comandos / impl.	USER, PASS, LIST , RETR n , DELE n (só marca; efetiva no QUIT); respostas +OK/−ERR; Qpopper, dovecot	—

Diálogos via telnet (prática da aula 6)

```
# SMTP telnet IP 25 → 220 ... ESMTP Exim
HELO exemplo.com.br → 250 · MAIL FROM: <martins@exemplo.com.br> → 250 OK · RCPT TO: <user@exemplo.com.br> → 250 Accepted
DATA → 354 ... From:/To:/Subject: + corpo + "." → 250 OK · QUIT → 221      # conferir: cat /var/mail/user
# POP3 telnet IP 110 → +OK Dovecot ready · USER user · PASS user → +OK Logged in · LIST → 1 778 · RETR 1 · DELE 1 →
Marked · QUIT
```

Provável na prova

- **L5 q.1 (quase certa)**: os três passos do MTA (MX no DNS → ordem de preferência crescente → armazenar na caixa).
- **L5 q.2**: dois servidores porque SMTP (25) **envia/encaminha** e POP3 (110)/IMAP (143) **acessa a caixa**.
- **L5 q.3**: anexos = **MIME**, camada de **apresentação**. **L5 q.4**: SMTP vira cliente ao encaminhar ao MTA de destino (e ao consultar o DNS).
- POP3 × IMAP (portas + 3 diferenças); ordem HELO → MAIL FROM → RCPT TO → DATA → "." → QUIT.

10 · Web: HTTP, URL e Proxy

1990: Tim Berners-Lee cria a WorldWideWeb para documentos científicos (hipertexto em **HTML**); boom com os ISPs (~1994); padrões controlados pelo **W3C**; 2000: Web Semântica (XML, RDF). Hipermídia (figuras, áudio, vídeo) é referenciada com o **MIME** — a ponte com o correio.

URL — RFC 1738

```
<protocol>://[<usr>:<pass>@]<host>[:<port>]/[<url-path>][?<par>=<value>&...]
```

Obrigatórios: protocol (http, https, ftp) e host (FQDN ou IP). Opcionais: user:pass, port (omitida = a do protocolo). Pares var=value após ?, separados por &, levam dados de uma request (**GET**). Ex.:

http://www.google.com.br/search?hl=pt-BR&q=teste → http (porta 80), host www.google.com.br, path /search, parâmetros hl e q.

HTTP — RFC 2616 (HTTP/1.1)

Método	Solicita ao servidor
GET	Enviar a página referenciada pela URL (pode ter parâmetros)
PUT	Receber um documento (tipo MIME no Content-Type) e armazená-lo na URL
POST	Receber dados no conteúdo e informá-los a um programa na URL (formulários)

Headers citados: Content-Encoding, Content-Language, Content-Length, Date, Content-Type (MIME, RFC 822).

Request = ASCII, linhas separadas por linha em branco, 1ª linha = método + path. **Response** = conteúdo MIME.

Servidor na porta **80** (Apache, IIS); programas server-side (CGI, PHP) geram conteúdo dinâmico.

Um recurso = uma conexão TCP

Cada requisição HTTP pede um recurso hipermídia e **normalmente gera uma nova conexão TCP, encerrada na resposta**. Página com 40 elementos (1 HTML + 30 ícones + 8 imagens + 1 CSS) = 40 conexões: 40 handshakes (~280 segmentos de controle), 40 sockets/buffers/entradas na tabela do SO, ≥ 1 RTT antes do 1º byte de cada. Com 100 usuários, 4.000 conexões. **O custo é por conexão, não por byte**: 30 ícones de 2 KB pesam mais que uma imagem de 60 KB.

Caching: cliente e proxy

	Cache local (navegador)	Servidor Proxy (Squid, porta 3128)
Onde / quem ganha	Na máquina do cliente; só aquele usuário	Na borda da rede local; todos os clientes da rede
Funcionamento	Guarda cópias das páginas recentes e recarrega do disco	Clientes pedem tudo ao proxy; ele busca no servidor Web, guarda cópia com URL + timestamp e reaproveita para qualquer cliente enquanto não expirar (mesmo papel do TTL no DNS)
Controle	Nenhum	ACLs (Access Control Lists) por protocolo, origem/destino e usuário — <code>/etc/squid/squid.conf</code> ; junto ao firewall aplica política de acesso à Internet

Provável na prova

- **L6 q.1 (quase certa)**: muitos elementos pequenos sobrecarregam porque cada recurso = uma requisição e uma conexão TCP (handshake, socket, buffers, tabela do SO); custo por conexão, não por byte; multiplicado pelos clientes simultâneos.
- **L6 q.2 (quase certa)**: proxy **próximo aos clientes**: (a) o cache só economiza banda antes do trecho caro (junto ao servidor, o tráfego já cruzou a Internet); (b) ACLs e política de acesso só fazem sentido onde estão os clientes, junto ao firewall.
- Decompor uma URL; GET × PUT × POST ("armazena na URL" × "informa dados a um programa"); portas 80 e 3128.

Laboratório: comandos essenciais (VM Debian)

Comando	Responde a pergunta	Aula
<code>ip -4 a / ifconfig · route -n</code>	Qual é o meu IP? Para onde meus pacotes vão?	3
<code>nmap <ip> · nmap -sU <ip></code>	Que portas TCP / UDP estão abertas naquela máquina? (UDP é lento: sem handshake)	3
<code>cat /etc/services</code>	Que serviço usa a porta N? (base da IANA)	3
<code>netstat -na --inet · lsof -i tcp / -i udp</code>	Que portas do meu sistema estão ativas? Qual processo segura cada socket? (socket é arquivo)	3

tcptraceroute · wireshark	Caminho de um segmento TCP · o que trafega no fio	3
ssh user@IP · telnet IP [porta] · ftp user@IP	Terminal cifrado · terminal em claro / cliente de protocolo texto · sessão FTP (ls, get, delete, quit)	4, 6
host -t MX <domínio> · host <nome> / host <ip>	Mail exchangers · resolução direta (A) e reversa (PTR)	5
cat /var/mail/user	O que chegou na caixa postal local?	6

Sequência de configuração que se repete

```
# preparação: su - · service NetworkManager stop · service networking stop · apt-get install net-tools nmap wireshark
# interface e rota (N = n° da máquina): ifconfig enp0s3 192.168.N.1/24 up · route add -net 192.168.N.0/24 enp0s3 · route
add default gw 192.168.N.1
# servidores via inetd (aula 4): apt-get install ssh inetutils-telnetd inetutils-ftpd ftp → descomentar telnet/ftp em
/etc/inetd.conf → service inetutils-inetd stop/start
# DNS (aula 5): apt-get install bind9 → zonas em named.conf.local + db.exemplo.com.br + db.N.168.192 → service named
stop/start
# correio (aula 6): apt-get install exim4 (update-exim4.conf.conf: configtype='internet', relay_domains, relay_nets) ·
dovecot-pop3d (auth_allow_cleartext=yes só para testar; desligar depois)
```

Erros comuns: nome da interface errado (é enp0s3/enp0s8); route -n vazio → adicionar a rota; IP some → NetworkManager reassumiu; conexão recusada no telnet/ftp → linha comentada no inetd.conf ou inetd não reiniciado; BIND mudo → sintaxe da zona ou serial não incrementado; e-mail "frozen" no exim → MX aponta para a própria máquina.

Números, portas e RFCs que caem

Puro decoreba em uma página. Cinco minutos antes da prova, leia só isto.

Porta	Serviço	Obs.	RFC	Define
20	FTP dados	abre/fecha a cada transferência	768 / 793	UDP / TCP
21	FTP controle	aberta a sessão toda	822	Formato de mensagem de e-mail (reusado pelo HTTP)
22	SSH	também túnel	854 / 959	Telnet / FTP
23	Telnet	texto plano	1034 e 1035	DNS (1034 = tipos de registro)
25	SMTP	clientes e outros MTAs	1057 / 1889	RPC / RTP
53	DNS	servidor de nomes	1738 / 2616	URL / HTTP 1.1
80	HTTP	Well Known	1939 / 3501	POP3 / IMAP v4
110	POP3	baixa e remove	2045-2049 / 2821	MIME / SMTP
143	IMAP	estado no servidor	3232	Portas IANA
3128	Proxy Squid	> 1023	4250-4254	SSH
3306	MySQL	Registered		

Grandeza	Valor
Campo de porta (TCP e UDP)	16 bits → 65.536 valores
Endereço IPv4 · SEQ e ACK do TCP	32 bits → $2^{32} = 4.294.967.296$
Header UDP · header TCP	8 bytes (4×16 bits) · 20 bytes mínimo, 60 máximo (Data Offset $\times 4$)
Payload máximo do IP · MTU Ethernet · MSS	65.515 bytes · 1.500 bytes · 1.460 bytes
TTL de registro DNS	inteiro de 32 bits, em segundos

As fórmulas

$$RTT_{\text{new}} = (\alpha \times RTT_{\text{old}}) + ((1 - \alpha) \times RTT_{\text{sample}})$$

$$RTO = \beta \times RTT_{\text{new}}$$

$$\text{UDP length} = 8 + \text{dados}$$

$$\text{header TCP} = \text{Data Offset} \times 4$$

Exemplo de bolso: α 0,875 · RTT_{old} 100 · RTT_{sample} 140 · β 2 → $RTT_{\text{new}} = 87,5 + 17,5 = 105 \text{ ms}$ → $RTO = 210 \text{ ms}$.

O que NÃO cai e onde o guia inferiu

Item da ementa	Situação
Transporte: UDP e TCP · DNS, Telnet, SSH, FTP, SMTP, POP/IMAP, proxy	Núcleo — slide + prática + exercícios (módulos 1-10)
Interface de sockets	Lacuna parcial — sem slide dedicado; só "SAP = sockets", as 5 primitivas, o diagrama da lousa e <code>gethostbyname</code> . Confirme com o professor
HTTPS	Lacuna — citado de passagem na URL; nenhum slide sobre TLS
Samba e CUPS · Segurança de rede · Criptografia como tema	Fora — sem slide, prática ou exercício até a P1 (o que caiu foi o uso aplicado no SSH)

- **Fato do material:** definições, campos, comandos, portas, RFCs, as duas fórmulas, os diálogos SMTP/POP3 e os arquivos de zona.
- **Inferência do guia:** os formatos de questão (derivados das listas); esclarecimentos como persist timer, iterativa × recursiva, MSS 1460, sincronização global, ECN e keep-alive; os valores numéricos dos exemplos (α 0,875, $x = 1000/y = 5000$, os 40 recursos). Só a janela deslizante (4K, WIN 2048/0/2048) veio pronta do slide. Se a questão pedir a definição do professor, responda pelo slide.
- **FTP passivo:** responda pelo slide — o cliente abre conexão com a **porta 20** do servidor.

Cobertura dos exercícios: aula 1 (11), aula 2 (10), aula 4 (3), aula 5 (4), aula 6 (2) = 30 questões, todas respondidas nos blocos "Provável na prova".